

Лабораторна робота №4

MongoDB та Mongoose: міграція REST API на базу даних

Мета роботи: набуття практичних навичок роботи з MongoDB та Mongoose шляхом міграції REST API з лабораторної роботи №3 з in-memory сховища на повноцінну базу даних

3. Завдання

3.1. Завдання 1: MongoDB Atlas та MongoDB Compass

Зареєструйтесь у хмарному сервісі MongoDB та підключіться до нього через GUI-клієнт.

Вимоги:

- Створіть акаунт на cloud.mongodb.com та безкоштовний кластер M0.
- У Network Access додайте IP 0.0.0.0/0 (дозволити з будь-якого місця — для розробки).
- Створіть Database User з паролем, збережіть рядок підключення формату `mongodb+srv://....`
- Встановіть MongoDB Compass та підключіться до кластера за отриманим рядком.
- Через Compass вручну створіть базу даних і колекцію, що відповідає вашій сутності з лаб. 3.
- Додайте кілька документів вручну через Compass — переконайтесь, що вони відображаються.
- Створіть файл `.env` у корені проекту:

```
MONGODB_URI=mongodb+srv://user:password@cluster.mongodb.net/mydb
PORT=3000
```

- Додайте `.env` до `.gitignore` та створіть `.env.example` з шаблоном (без реальних паролів).

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.21.121.23.000 – Лр.4		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Сорока А.О.			Звіт з лабораторної роботи №4	Літ.	Арк.
Перевір.		Фуріхата Д. В.					1
Реценз.						ФІКТ, гр. ВТ-23-1	
Н. Контр.							
Зав.каф.							

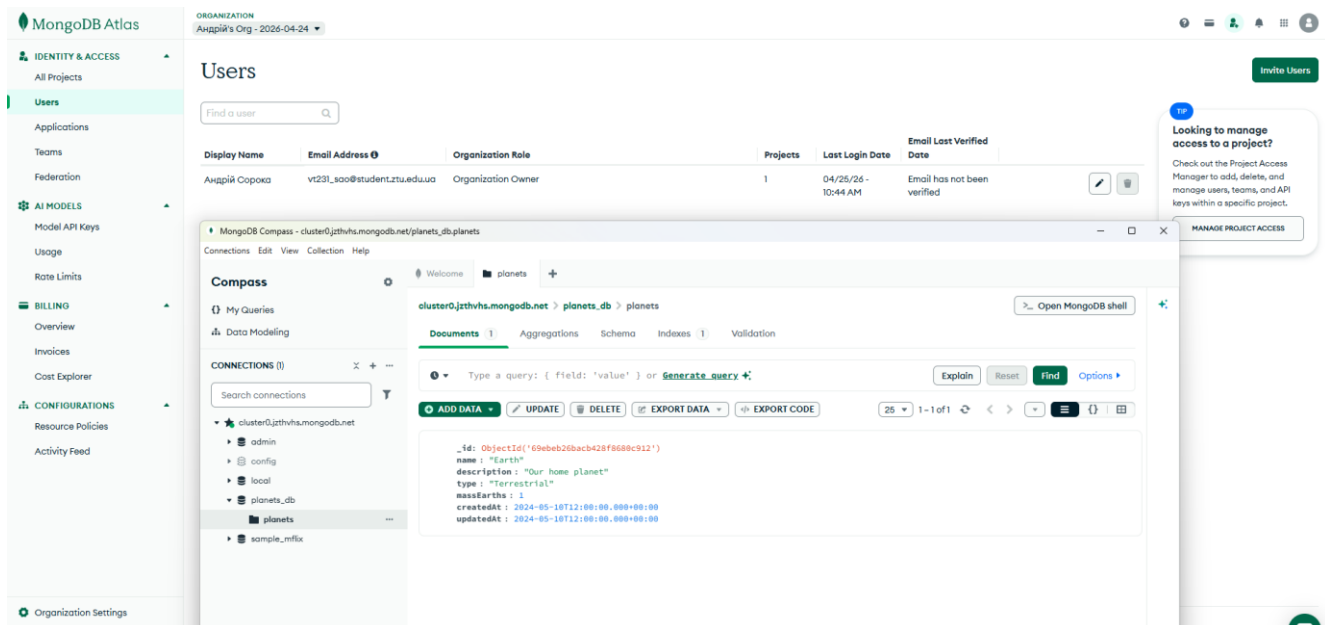


Рис. 1. Створення акаунт і підключення

3.2. Завдання 2: Підключення Mongoose та оновлення server.ts

Встановіть Mongoose та налаштуйте підключення до бази даних перед стартом сервера.

Вимоги:

- Встановіть mongoose.
- Встановіть dotenv та підключіть його на початку server.ts.
- Створіть src/config/database.ts з асинхронною функцією підключення до бази даних.
 - Зчитує MONGODB_URI зі змінних середовища.
 - Реєструє обробники подій помилки та відключення на об'єкті з'єднання Mongoose.
- Оновіть server.ts: спочатку підключіться до бази даних, і лише потім запустіть сервер. При помилці підключення — завершіть процес з кодом помилки.
- Додайте graceful shutdown: обробіть сигнал SIGTERM і закрийте з'єднання з базою даних перед виходом.

Файл: src/config/database.ts

```
import mongoose from 'mongoose';

export async function connectDB(): Promise<void> {
  const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017/planets_db';

  try {
    await mongoose.connect(uri);
    console.log('Успішне підключення до MongoDB');
  } catch (error) {
    console.error('Помилка підключення до MongoDB:', error);

    process.exit(1);
  }
}

mongoose.connection.on('error', (err) => {
  console.error('MongoDB помилка під час роботи:', err);
});

mongoose.connection.on('disconnected', () => {
  console.warn('MongoDB відключено');
});
```

Файл: src/server.ts

```
import dotenv from 'dotenv';

dotenv.config();

import app from './app';
import { connectDB } from './config/database';
import mongoose from 'mongoose';

const PORT = process.env.PORT || 3000;

async function startServer() {
  await connectDB();

  const server = app.listen(PORT, () => {
    console.log(`Сервер успішно запущено на порту ${PORT}`);
  });

  process.on('SIGTERM', gracefulShutdown);
  process.on('SIGINT', gracefulShutdown);

  function gracefulShutdown(signal: string) {
    console.log(`\nОтримано ${signal}. Граційне завершення роботи...`);
    server.close(async () => {
      console.log('HTTP сервер зупинено.');
```

3.3. Завдання 3: Mongoose-схема та модель

Створіть Mongoose-схему для вашої сутності, яка відображає ті самі атрибути, що й Zod-схема.

Вимоги:

- Створіть src/models/{entity}.model.ts.
- Оголосіть TypeScript-інтерфейс для вашої сутності з усіма її полями (без _id — Mongoose додає його автоматично).
- Опишіть Mongoose-схему на основі цього інтерфейсу з увімкненим автоматичним управлінням датами (timestamps: true). Для кожного поля вкажіть тип, обов'язковість, перелік дозволених значень, дефолтне значення та обмеження на довжину/розмір тощо — відповідно до Zod-схеми з лаб. 3.
- Zod-схеми залишаються незмінними — вони валідують HTTP-запити, Mongoose-схема забезпечує цілісність у БД.
- Реалізуйте щонайменше 1 кастомний валідатор для одного з полів — функція-предикат з описовим повідомленням про помилку.
- Реалізуйте щонайменше 1 віртуальну властивість — computed поле, що не зберігається в БД, але обчислюється на основі наявних полів. Увімкніть включення віртуальних властивостей у JSON-серіалізацію, щоб вони потрапляли до API-відповідей.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

Файл: src/models/planet.model.ts

```
import mongoose, { Schema, Document } from 'mongoose';

export interface IPlanet extends Document {
  name: string;
  description?: string;
  type: 'Terrestrial' | 'Gas Giant' | 'Ice Giant' | 'Dwarf Planet';
  massEarth: number;
  // Віртуальне поле
  massKg: string;
  createdAt: Date;
  updatedAt: Date;
}

const PlanetSchema: Schema = new Schema(
  {
    name: {
      type: String,
      required: [true, 'Назва планети є обов\'язковою'],
      unique: true,
      trim: true,
      minlength: [1, 'Назва занадто коротка'],
      maxlength: [100, 'Назва занадто довга']
    },
    description: {
      type: String,
      maxlength: [500, 'Опис не може перевищувати 500 символів']
    },
    type: {
      type: String,
      required: true,
      enum: {
        values: ['Terrestrial', 'Gas Giant', 'Ice Giant', 'Dwarf Planet'],
        message: 'Тип {VALUE} не підтримується'
      }
    },
    massEarth: {
      type: Number,
      required: true,
      // КАСТОМНИЙ ВАЛІДАТОР (Завдання 4)
      validate: {
        validator: function (value: number) {
          if (this.type === 'Dwarf Planet' && value >= 0.1) {
            return false;
          }
          return value > 0;
        },
        message: 'Маса має бути > 0. Для Dwarf Planet маса має бути < 0.1 Землі'
      }
    },
    timestamps: true,
    toJSON: {
      virtuals: true,
      transform: function (doc, ret: Record<string, any>) {
        ret.id = ret._id.toString();
        delete ret._id;
        delete ret.__v;
        return ret;
      },
      virtuals: true
    },
    toObject: { virtuals: true }
  }
);

PlanetSchema.virtual('massKg').get(function (this: IPlanet) {
  const earthMassKg = 5.972e24;
  const totalMass = this.massEarth * earthMassKg;
  return `${totalMass.toExponential(2)} kg`;
});

export const Planet = mongoose.model<IPlanet>('Planet', PlanetSchema);
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

3.4. Завдання 4: Заміна сховища на Mongoose

Замініть Мар-сховище з лаб. 3 на виклики Mongoose Model, зберігши той самий API-контракт.

Вимоги:

- Замініть весь вміст storage-модуля (src/storage/{entity}.ts) на функції, що використовують EntityModel.
- MongoDB генерує `_id` (ObjectId) автоматично — ручна генерація ідентифікатора та конструювання об'єкта більше не потрібні.
- Кожна операція реалізується відповідним методом моделі. При оновленні переконайтесь, що метод повертає **оновлений** документ і запускає валідатори схеми.
- Збережіть (або розширте за необхідності) ті самі статус-коди відповідей: 200, 201, 204, 400, 404.
- Функцію скидання сховища (reset) можна видалити — в тестах використовуватиметься очищення через модель.

src/storage/planet.storage.ts

```
import { Planet, IPlanet } from '../models/planet.model';
import { CreatePlanetInput, UpdatePlanetInput } from '../schemas/planet.schema';

export interface PlanetFilters {
  type?: string;
  minMassEarth?: number;
  page?: number;
  limit?: number;
  sortBy?: string;
}

export const PlanetStorage = {
  async getAll(filters?: PlanetFilters): Promise<IPlanet[]> {
    const query: any = {};

    if (filters?.type) query.type = filters.type;
    if (filters?.minMassEarth !== undefined) {
      query.massEarth = { $gte: Number(filters.minMassEarth) }; // gte >=
    }

    let mongooseQuery = Planet.find(query);

    if (filters?.sortBy) {
      mongooseQuery = mongooseQuery.sort(filters.sortBy);
    } else {
      mongooseQuery = mongooseQuery.sort('-createdAt');
    }

    // Пагінація
    const page = filters?.page ? Number(filters.page) : 1;
    const limit = filters?.limit ? Number(filters.limit) : 10;
    const skip = (page - 1) * limit;

    mongooseQuery = mongooseQuery.skip(skip).limit(limit);

    return await mongooseQuery.exec();
  },

  async getById(id: string): Promise<IPlanet | null> {
    return await Planet.findById(id).exec();
  },
}
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		5

```

    async create(data: CreatePlanetInput): Promise<IPlanet> {
      const newPlanet = new Planet(data);
      return await newPlanet.save();
    },

    async update(id: string, data: UpdatePlanetInput): Promise<IPlanet | null> {
      return await Planet.findByIdAndUpdate(id, data, { new: true, runValidators: true }).exec();
    },

    async delete(id: string): Promise<boolean> {
      const result = await Planet.findByIdAndDelete(id).exec();
      return result !== null;
    },
  },
};

```

3.5. Завдання 5: Обробка помилок MongoDB

Розширте errorHandler для обробки специфічних помилок Mongoose та MongoDB.

Вимоги:

- Розширте `src/middleware/errorHandler.ts`, щоб правильно обробляти специфічні помилки Mongoose та MongoDB з відповідними статус-кодами. Обробку помилок Zod з лаб. 3 збережіть.
- Переконайтеся, що запит до неіснуючого, але валідного ObjectId повертає 404, а запит з невалідним форматом ID — 400.

Файл: `src/middleware/error.middleware.ts`

```

import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';
import mongoose from 'mongoose';

export const errorHandler = (err: any, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof ZodError) {
    return res.status(400).json({
      status: 'error',
      message: 'Помилка валідації даних',
      errors: err.issues
    });
  }

  if (err.code === 11000) {
    return res.status(409).json({
      status: 'error',
      message: 'Запис з такими унікальними даними вже існує'
    });
  }

  if (err instanceof mongoose.Error.CastError) {
    return res.status(400).json({
      status: 'error',
      message: `Невірний формат ідентифікатора для поля ${err.path}`
    });
  }

  if (err instanceof mongoose.Error.ValidationError) {
    const messages = Object.values(err.errors).map(val => val.message);
    return res.status(400).json({
      status: 'error',
      message: 'Помилка валідації бази даних',
      errors: messages
    });
  }
}

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		


```

console.error('Unhandled error:', err);
return res.status(500).json({
  status: 'error',
  message: 'Внутрішня помилка сервера'
});
};

```

3.6. Завдання 6: Фільтрація, сортування та пагінація

Переробіть фільтрацію з лаб. 3 на Mongoose Query API та додайте сортування і пагінацію.

Вимоги:

- Збережіть щонайменше 2 query-параметри фільтрації з лаб. 3, переробивши логіку з in-memoгу фільтрації на умови Mongoose-запиту — використовуйте оператори порівняння, регулярні вирази або перелік значень залежно від типу поля.
- Додайте сортування через query-параметр sort. Визначте конвенцію для позначення напрямку сортування та реалізуйте її послідовно.
- Реалізуйте пагінацію з параметрами page та limit зі зрозумілими значеннями за замовчуванням. Підрахунок загальної кількості документів та вибірку виконуйте **паралельно**.
- Формат відповіді GET /api/items змінюється: замість плоского масиву — об'єкт із полями data та pagination (поточна сторінка, ліміт, загальна кількість, кількість сторінок).
- Збережіть додатковий доменний маршрут з лаб. 3, адаптувавши його для Mongoose.

Фільтрація в файлі (src/storage/planet.storage.ts)

```

async getAll(filters?: PlanetFilters): Promise<IPланет[]> {
  const query: any = {};

  if (filters?.type) query.type = filters.type;
  if (filters?.minMassEarth !== undefined) {
    query.massEarth = { $gte: Number(filters.minMassEarth) }; // gte >=
  }

  let mongooseQuery = Planet.find(query);

  if (filters?.sortBy) {
    mongooseQuery = mongooseQuery.sort(filters.sortBy);
  } else {
    mongooseQuery = mongooseQuery.sort('-createdAt');
  }

  // Пагінація
  const page = filters?.page ? Number(filters.page) : 1;
  const limit = filters?.limit ? Number(filters.limit) : 10;
  const skip = (page - 1) * limit;

  mongooseQuery = mongooseQuery.skip(skip).limit(limit);

  return await mongooseQuery.exec();
},

async getById(id: string): Promise<IPланет | null> {
  return await Planet.findById(id).exec();
},

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		

3.7. Завдання 7: Тестування з mongodb-memory-server

Перепишіть тести для роботи з реальною MongoDB у пам'яті замість in-memory Map.

Вимоги:

- Встановіть mongodb-memory-server як dev-залежність.
- Створіть tests/setup.ts з допоміжними функціями запуску і зупинки тестової БД.
- У кожному тестовому файлі використовуйте хуки beforeAll/afterAll для підключення та afterEach для очищення колекції між тестами.
- Напишіть **unit-тести моделі** (в окремому блоці describe) — перевірте дефолти, автоматичні дати, правила валідації, віртуальні властивості тощо.
- Перепишіть **інтеграційні тести** з лаб. 3 для роботи з MongoDB — покрийте всі маршрути, включаючи граничні випадки специфічні для MongoDB.
- Фінальна кількість тестів — не менша за лаб. 3.
- Спробуйте досягти 100% покриття тестами коду, що стосується логіки проекту.

Файл: tests/setup.ts

```
import mongoose from 'mongoose';
import { MongoMemoryServer } from 'mongodb-memory-server';

let mongoServer: MongoMemoryServer;

// 1. Запуск сервера і підключення (викликається ОДИН раз перед усіма тестами)
export const connectDBForTesting = async () => {
  mongoServer = await MongoMemoryServer.create();
  const uri = mongoServer.getUri();
  await mongoose.connect(uri);
};

// 2. Зупинка сервера (викликається ОДИН раз після всіх тестів)
export const closeDBForTesting = async () => {
  await mongoose.connection.dropDatabase();
  await mongoose.connection.close();
  await mongoServer.stop();
};

// 3. Очищення даних (викликається ПЕРЕД КОЖНИМ тестом)
export const clearDBForTesting = async () => {
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    const collection = collections[key];
    await collection.deleteMany({});
  }
};
```

Файл: tests/planet.model.test.ts

```
import { Planet } from '../models/planet.model';
import { connectDBForTesting, closeDBForTesting, clearDBForTesting } from './setup';

describe('Planet Mongoose Model Unit Tests', () => {

  beforeAll(async () => await connectDBForTesting());
  afterEach(async () => await clearDBForTesting());
  afterAll(async () => await closeDBForTesting());

  it('повинна успішно створити валідну планету', async () => {
    const planet = new Planet({
      name: 'Earth',
      type: 'Terrestrial',
      massEarth: 1
    });
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		


```

    });
    const savedPlanet = await planet.save();

    expect(savedPlanet._id).toBeDefined();
    expect(savedPlanet.name).toBe('Earth');
    expect(savedPlanet.createdAt).toBeDefined();
    expect(savedPlanet.updatedAt).toBeDefined();
  });

  it('повинна правильно рахувати віртуальне поле massKg', async () => {
    const planet = new Planet({
      name: 'Jupiter',
      type: 'Gas Giant',
      massEarth: 317.8
    });
    const savedPlanet = await planet.save();

    expect(savedPlanet.massKg).toBeDefined();
    expect(savedPlanet.massKg).toContain('kg');
  });

  it('повинна видати помилку валідації, якщо не вказано обов'язкове поле (name)', async () => {
    const planet = new Planet({ type: 'Gas Giant', massEarth: 10 });
    let err: any;
    try {
      await planet.save();
    } catch (error) {
      err = error;
    }
    expect(err).toBeDefined();
    expect(err.errors.name).toBeDefined();
  });

  it('повинна спрацювати кастомна валідація для Dwarf Planet (маса < 0.1)', async () => {
    const pluto = new Planet({
      name: 'Pluto',
      type: 'Dwarf Planet',
      massEarth: 0.5
    });
    let err: any;
    try {
      await pluto.save();
    } catch (error) {
      err = error;
    }
    expect(err).toBeDefined();
    expect(err.errors.massEarth.message).toContain('Для Dwarf Planet маса має бути < 0.1');
  });
});

```

Файл: tests/planet.test.ts

```

import request from 'supertest';
import app from '../app';
import { connectDBForTesting, closeDBForTesting, clearDBForTesting } from './setup';

const validPlanet = {
  name: 'Earth',
  description: 'Our home',
  type: 'Terrestrial',
  massEarth: 1
};

const heavyPlanet = {
  name: 'Jupiter',
  type: 'Gas Giant',
  massEarth: 317.8
};

describe('Planet REST API Integration Tests (MongoDB)', () => {
  beforeAll(async () => await connectDBForTesting());
  afterEach(async () => await clearDBForTesting());
  afterAll(async () => await closeDBForTesting());

  describe('POST /api/planets', () => {
    it('повинен створити нову планету і повернути статус 201', async () => {
      const res = await request(app).post('/api/planets').send(validPlanet).expect(201);
      expect(res.body).toHaveProperty('id');
      expect(res.body.name).toBe(validPlanet.name);
      expect(res.body).toHaveProperty('massKg');
    });
  });
});

```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    it('повинен повернути 409 (Conflict), якщо планета з таким іменем вже існує', async () => {
      await request(app).post('/api/planets').send(validPlanet).expect(201);
      const res = await request(app).post('/api/planets').send(validPlanet).expect(409);
      expect(res.body.message).toContain('Duplicate Key');
    });
  });

describe('GET /api/planets', () => {
  it('повинен повернути порожній масив, якщо база чиста', async () => {
    const res = await request(app).get('/api/planets').expect(200);
    expect(res.body).toEqual([]);
  });

  it('повинен повернути всі створені планети з підтримкою пагінації', async () => {
    await request(app).post('/api/planets').send({ ...validPlanet, name: 'P1' });
    await request(app).post('/api/planets').send({ ...validPlanet, name: 'P2' });

    const res = await request(app).get('/api/planets?limit=1').expect(200);
    expect(res.body).toHaveLength(1);
  });
});

describe('GET /api/planets/:id', () => {
  it('повинен повернути планету за правильним ID', async () => {
    const createRes = await request(app).post('/api/planets').send(validPlanet);
    const res = await request(app).get(`/api/planets/${createRes.body.id}`).expect(200);
    expect(res.body.id).toBe(createRes.body.id);
  });

  it('повинен повернути 400 (Bad Request), якщо формат ID невалідний для MongoDB', async () => {
    const res = await request(app).get('/api/planets/invalid-mongo-id-123').expect(400);
    expect(res.body.message).toContain('Невірний формат ідентифікатора');
  });

  it('повинен повернути 404, якщо ID валідний, але планети немає', async () => {
    await request(app).get('/api/planets/507flf77bcf86cd799439011').expect(404);
  });
});

describe('PATCH /api/planets/:id', () => {
  it('повинен успішно оновити лише передані поля', async () => {
    const createRes = await request(app).post('/api/planets').send(validPlanet);
    const res = await request(app)
      .patch(`/api/planets/${createRes.body.id}`)
      .send({ name: 'Earth 2.0' })
      .expect(200);
    expect(res.body.name).toBe('Earth 2.0');
  });
});

describe('DELETE /api/planets/:id', () => {
  it('повинен видалити планету і повернути статус 204 No Content', async () => {
    const createRes = await request(app).post('/api/planets').send(validPlanet);
    await request(app).delete(`/api/planets/${createRes.body.id}`).expect(204);
    await request(app).get(`/api/planets/${createRes.body.id}`).expect(404);
  });
});
});

```

```

$ npm run test:coverage
PS D:\university\Node.js\lab_03>
> lab_03@1.0.0 test:coverage
> jest --coverage

PASS src/tests/planet.model.test.ts (7.656 s)
(node:3092) [MongoDB] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
(Use 'node --trace-warnings...' to show where the warning was created)
PASS src/tests/planet.test.ts (8.019 s)

-----
File | % Stats | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 85.07 | 64.7 | 90.47 | 85.82 |
src | 100 | 100 | 100 | 100 |
app.ts | 100 | 100 | 100 | 100 |
src/middleware | 65.21 | 50 | 75 | 65 |
error.middleware.ts | 56.25 | 50 | 50 | 57.14 | 9,33-44
validate.middleware.ts | 85.71 | 100 | 100 | 83.33 | 11
src/models | 100 | 100 | 100 | 100 |
planet.model.ts | 100 | 100 | 100 | 100 |
src/routes | 79.06 | 66.66 | 83.33 | 79.06 |
planet.routes.ts | 79.06 | 66.66 | 83.33 | 79.06 | 12-17,37,65,69,77,81
src/schemas | 100 | 100 | 100 | 100 |
planet.schema.ts | 100 | 100 | 100 | 100 |
src/storage | 86.36 | 60 | 100 | 90.47 |
planet.storage.ts | 86.36 | 60 | 100 | 90.47 | 18,24
src/tests | 100 | 100 | 100 | 100 |
setup.ts | 100 | 100 | 100 | 100 |
-----

Test Suites: 2 passed, 2 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 9.096 s, estimated 10 s
$ npm run test:coverage
PS D:\university\Node.js\lab_03>

```

Рис. 2. Результат тестування

					Арк.
					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.121.23.000 – Лр.4
Змн.	Арк.	№ докум.	Підпис	Дата	10